

Simulating Probability Distributions and Stochastic Processes

Jenny Petrova

March 2026

Contents

1	Executive Summary	2
2	Objectives	2
3	Methods for Simulating Non-Uniform Distributions	2
3.1	Acceptance-Rejection Method	2
3.1.1	Numerical Example	3
3.2	Box–Muller Method	5
3.3	Beasley–Springer–Moro Algorithm	5
3.3.1	Numerical Example	6
3.4	Inverse Transformation Method	8
3.4.1	Numerical Example	8
3.5	General Sampling Method	9
3.5.1	Numerical Example	9
4	Methods for Simulating Stochastic Processes	10
4.1	Brownian Bridge Construction	10
4.1.1	Numerical Example	11
5	Conclusion	12
6	Complete Code	13

1 Executive Summary

We previously examined random number generators for producing samples from the uniform distribution, along with statistical tests to assess the quality of the generators. In practice, however, it is often necessary to simulate samples from non-uniform distributions. This report explores several transformation techniques for generating such samples of non-uniform random variables. We also discuss procedures for simulating paths of continuous-time stochastic processes. Numerical examples are provided, and the implementation details of each method are analyzed.

2 Objectives

We describe and construct numerical examples of the following Monte Carlo simulation methods:

- Inverse transformation method
- Acceptance-rejection method
- Box-Muller method
- Generating Brownian Motion (Wiener Process) recursively

3 Methods for Simulating Non-Uniform Distributions

The following methods transform a random sample from the uniform distribution to a sample from another probability distribution.

3.1 Acceptance-Rejection Method

The **Acceptance-Rejection Method** is used to simulate samples from a target distribution with probability density function $f(x)$ using a more convenient distribution with density $g(x)$ from which we know how to generate samples. The idea is to generate candidate samples X from g until you can *accept* a sample from f with probability $f(X)/cg(X)$. In this design, c is a constant that satisfies

$$\frac{f(X)}{g(X)} \leq c \tag{1}$$

for all X , hence $f(X)/cg(X)$ will be a value between 0 and 1. The accepted samples are therefore distributed according to the target density $f(x)$.

3.1.1 Numerical Example

Chapter 3 Problem 5

Describe in detail how to simulate the value of a random variable X such that

$$P\{X = 1\} = 0.3$$

$$P\{X = 2\} = 0.2$$

$$P\{X = 3\} = 0.35$$

$$P\{X = 4\} = 0.15$$

using the acceptance-rejection algorithm. What is the average complexity of the algorithm? In other words, find $\mathbb{E}[T]$ where T is the number of arithmetical operations ($+$, $-$, $\times$, $\div$) and comparisons (checking if a number is less than another) that need to be computed to simulate a value from X . (Ignore any operations in initialization and assignments $a := b$, and assume that generating a uniform random number is worth 3 arithmetical operations.)

To simulate the value of a random variable X using the acceptance-rejection algorithm, we generate a random variable Y on the same space as X . Let $f(i)$ denote the probabilities given above, for $i = 1, \dots, 4$. For this data we propose using a discrete uniform distribution on $(1, 4)$ to generate Y with PMF $g(i) = \frac{1}{4}$ for $i = 1, \dots, 4$. We fix an upper bound, c , to ensure $f(i)/g(i)$ is a probability between 0 and 1:

$$c = \max_{1 \leq i \leq 4} \frac{f(i)}{1/4} = \frac{0.35}{1/4} = 1.40.$$

Therefore $f(i)/cg(i) \in (0, 1)$. We can now design the following acceptance-rejection procedure:

1. Generate u_1 from $U(0, 1)$.
2. Set $y = \text{Floor}(4u_1) + 1$.
3. Generate u_2 from $U(0, 1)$.
4. If $u_2 \leq \frac{f(y)}{0.35}$, set $X = y$ and stop. Else, return to Step 1.

With each iteration, a simulated value y is accepted with probability $1/c$, hence $c = 1.4$ is the expected number of loops until we accept the sample. The average complexity of this algorithm is thus given by $\mathbb{E}[T] = c \cdot T = 1.4 \cdot T$. We obtain T by counting the operations at each step in the algorithm:

1. Generating a random variable from $U(0, 1)$ is done with an LCG. The recursion requires multiplication ($+1$) and addition ($+1$) at each step. The random variable is then scaled to a value between 0 and 1 with division ($+1$). **Total = 3 operations.**

2. Defining y requires multiplication (+1), flooring (+1), and addition (+1). **Total = 3 operations.**
3. Same as step 1. **Total = 3 operations.**
4. Checking the upper bound of u_2 requires division (+1) and comparison (+1). **Total = 2 operations.**

Summing the above operations, we have $T = 11$ for each loop in algorithm. Therefore the average complexity is $\mathbb{E}[T] = 1.4 \cdot 11 = 15.4$.

To verify the design of the Acceptance–Rejection method for the above parameters and data, we construct the following algorithm:

```

1  function acceptance_rejection()
2      f = [0.3, 0.2, 0.35, 0.15]
3      while true
4          u1 = rand()
5          u2 = rand()
6          y = Int(floor(4*u1)) + 1
7          if u2 <= f[y]/0.35
8              return y
9          end
10     end
11 end

```

We run 1000 iterations of this method to obtain 1000 samples from $f(i)$. The distribution of our samples is displayed in the figure below.

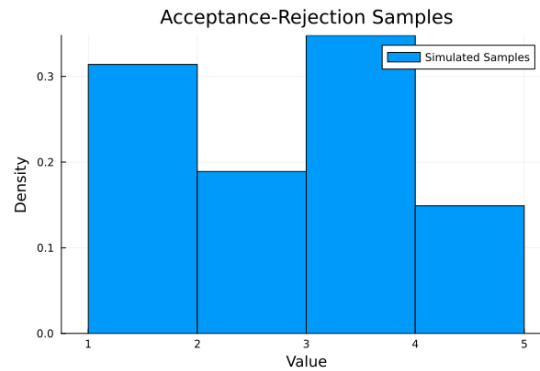


Figure 1: Histogram of 1000 acceptance-rejection samples

Observing the figure, the frequencies of the samples are nearly proportional to the corresponding probabilities of our target distribution. Therefore, the histogram confirms that the acceptance–rejection algorithm is correctly generating samples our target distribution.

3.2 Box–Muller Method

The **Box–Muller Method** is used to generate pairs of i.i.d. standard normal random variables from uniform random samples. The derivation of this method is based on a two-dimensional Cartesian system, where we consider the joint distribution of two independent standard normal random variables X and Y , and express them in polar coordinates. Let

$$R = \sqrt{X^2 + Y^2}, \quad \theta = \arctan\left(\frac{Y}{X}\right). \quad (2)$$

By transforming from Cartesian to polar coordinates, the joint density of (R, θ) can be shown to be

$$f_{R,\theta}(r, \theta) = \frac{1}{2\pi} e^{-r^2/2} \cdot r, \quad r \geq 0, \theta \in [0, 2\pi). \quad (3)$$

This factorizes as

$$f_{R,\theta}(r, \theta) = \left(r e^{-r^2/2} \right) \cdot \left(\frac{1}{2\pi} \right), \quad (4)$$

which implies that R and θ are independent, with

$$\theta \sim \text{Uniform}(0, 2\pi), \quad R^2 \sim \text{Exponential}\left(\frac{1}{2}\right).$$

Therefore, our implementation scheme is the following:

1. Generate $u_1, u_2 \sim U(0, 1)$.
2. Set $R = \sqrt{-2 \ln u_1}$ and $\theta = 2\pi u_2$.
3. Compute

$$X = R \cos \theta, \quad Y = R \sin \theta.$$

The final step transforms (R, θ) back to Cartesian coordinates, producing two independent standard normal random variables.

3.3 Beasley–Springer–Moro Algorithm

The **Beasley–Springer–Moro (BSM) algorithm** is a numerical method for generating standard normal samples by approximating the inverse cumulative distribution function $F^{-1}(u)$ of the standard normal distribution. If $u \sim U(0, 1)$, then

$$X = F^{-1}(u)$$

follows a standard normal distribution. Since F^{-1} has no closed-form expression, the BSM algorithm provides an accurate approximation. The procedure is the following:

1. Generate $u \sim U(0, 1)$. Compute $y = u - 0.5$.

2. If $|y| < 0.42$:

- i. Set $r = y^2$.
- ii. Approximate $F^{-1}(u)$ using a rational function:

$$X \approx y \cdot \frac{a_0 + a_1 r + a_2 r^2 + a_3 r^3}{1 + b_1 r + b_2 r^2 + b_3 r^3}.$$

3. If $|y| \geq 0.42$:

- i. Take

$$r = \begin{cases} u, & u < 0.5, \\ 1 - u, & u \geq 0.5. \end{cases}$$

- ii. Set $r = \ln(-\ln(r))$.
- iii. Approximate $F^{-1}(u)$ using a polynomial:

$$X \approx c_0 + c_1 r + c_2 r^2 + \cdots + c_8 r^8.$$

- iv. Enforce symmetry:

$$X = \begin{cases} X, & y \geq 0, \\ -X, & y < 0. \end{cases}$$

The coefficients a_i , b_i , and c_i above are predefined constants. The algorithm returns a random variable sampled from the standard normal distribution.

3.3.1 Numerical Example

Chapter 3 Problem 7

Write a computer program that implements the Box-Muller method and the Beasley-Springer-Moro algorithm (see pg. 67–68 of Glasserman's book). This algorithm gives an approximation to the inverse normal CDF. Then generate 2000 standard normals using this method. Test the normality of this data, and the Box-Muller data you generated in Problem 2(a), using the Anderson-Darling test. (You will need a table for the percentiles for the Anderson-Darling statistic.) Which data set is closer to the normal distribution?

We construct two methods for generating standard normal samples. Then, we test the normality of the generated sample data using the Anderson-Darling test statistic

$$A^2 = -N - \frac{1}{N} \sum_{i=1}^N \left((2i-1) \log Z_i + (2N+1-2i) \log(1-Z_i) \right),$$

where $Z_i = F(X_i)$ for ordered sample observations $X_1 \leq X_2 \leq \dots \leq X_N$. First, we implement the Box–Muller method:

```

1  function boxmuller()
2      u1 = rand()
3      u2 = rand()
4      r_squared = -2 * log(u1)
5      theta = 2 * pi * u2
6      z1 = sqrt(r_squared) * cos(theta)
7      z2 = sqrt(r_squared) * sin(theta)
8      return z1, z2
9  end

```

Running the Box–Muller method for 1000 iterations (hence producing $N = 2000$ samples) with seed 1234, we obtain an Anderson–Darling statistic $A_{BM}^2 \approx 0.763$. We also implement the Beasley–Springer–Moro algorithm:

```

1  function beasley_springer_moro()
2      a = [2.50662823884, -18.61500062529, 41.39119773534,
3          ↪ -25.44106049637]
4      b = [-8.47351093090, 23.08336743743, -21.06224101826,
5          ↪ 3.13082909833]
6      c = [0.3374754822726147, 0.9761690190917186, 0.1607979714918209,
7          ↪ 0.0276438810333863, 0.0038405729373609, 0.0003951896511919,
8          ↪ 0.0000321767881768, 0.0000002888167364, 0.0000003960315187]
9      u = rand(seed)
10     y = u - 0.5
11     if abs(y) < 0.42
12         r = y^2
13         x = y * (((a[4] * r + a[3]) * r + a[2]) * r + a[1]) /
14             ↪ (((b[4] * r + b[3]) * r + b[2]) * r + b[1]) * r + 1)
15     else
16         r = u
17         if y > 0
18             r = 1 - u
19             r = log(-log(r))
20             x = c[1] + r * (c[2] + r * (c[3] + r * (c[4] + r * (c[5]
21                 ↪ + r * (c[6] + r * (c[7] + r * (c[8] + r * c[9]))))))))
22         else
23             r = log(-log(r))
24             x = -(c[1] + r * (c[2] + r * (c[3] + r * (c[4] + r *
25                 ↪ (c[5] + r * (c[6] + r * (c[7] + r * (c[8] + r *
26                 ↪ c[9]))))))))
27         end
28     end
29     return x
30 end

```

After generating $N = 2000$ samples using the Beasley–Springer–Moro method with seed 1234, we obtain an Anderson–Darling statistic of $A_{BSM}^2 \approx 0.918$. We use the table below to compare the test statistics from both methods across varying significance levels:

Anderson–Darling Statistic	10%	5%	1%
A^2	1.933	2.492	3.857

Both methods yield test statistics below the critical values, so we fail to reject the null hypothesis of normality. Thus, both methods appear to produce samples consistent with a standard normal distribution. The slightly smaller value of A_{BSM}^2 this sample may be marginally closer to the normal distribution, although the difference may be due to sampling variability.

3.4 Inverse Transformation Method

Similar to the BSM algorithm, the **Inverse Transformation Method** generates random variables X by sampling from the inverse CDF F^{-1} such that

$$X = F^{-1}(u),$$

for $u \sim U(0, 1)$. This method works because

$$\mathbb{P}(X \leq x) = \mathbb{P}(F^{-1}(U) \leq x) = \mathbb{P}(U \leq F(x)) = F(x),$$

so X has distribution function F . The main limitation of this method is that it requires an explicit form of F^{-1} , which is not available for all probability distributions, motivating approximation methods such as the BSM algorithm we previously discussed.

3.4.1 Numerical Example

Chapter 3 Problem 8

Give a method for generating the Weibull distribution function

$$F(x) = 1 - e^{-\alpha x^\beta}, \quad 0 < x < \infty.$$

Let $u_1, u_2, \dots, u_n$ be i.i.d. random variables from the uniform distribution $U(0, 1)$. To generate random samples $X_1, X_2, \dots, X_n$ from the Weibull distribution function $F(x)$, we need to apply an inverse transformation method, taking $X_i = F^{-1}(u_i)$. Compute the inverse distribution function:

$$u = 1 - e^{-\alpha x^\beta} \implies e^{-\alpha x^\beta} = 1 - u \implies x = \left(-\frac{\ln(1 - u)}{\alpha} \right)^{1/\beta}.$$

Since $F : \mathbb{R} \mapsto [0, 1]$, then $1 - F = 1 - u$ is between 0 and 1. Therefore, we design the following scheme for simulating the Weibull distribution:

1. Generate u from $U(0, 1)$.

2. Set $X = \left(-\frac{\ln(u)}{\alpha}\right)^{1/\beta}$.

Repeat the above steps until n samples are obtained. We implement the inverse transformation algorithm to verify our implementation scheme:

```

1 function inverse_transform(alpha,beta)
2     u = rand()
3     return (-log(u) / alpha)^(1/beta)
4 end

```

Setting parameters $\alpha = 1$, $\beta = 5$, we generate 10 samples from the Weibull distribution:

```

1 [0.7018509853284556, 0.01290516975617361, 1.977052778128193,
  ↪ 0.005932466164874385, 0.2594572277871222, 0.37634412638432085,
  ↪ 0.7013506713717248, 0.8552969819869579, 0.19434732980405436,
  ↪ 0.7987542701229706]

```

3.5 General Sampling Method

In this section, we consider sampling from multivariate (or joint) probability distributions $F_1(x), F_2(x), \dots$. The joint CDF is given by

$$F_{X_1, \dots, X_N}(x_1, \dots, x_n) = P\{X_1 \leq x_1, \dots, X_N \leq x_N\}.$$

If the random variables $X_1, \dots, X_N$ are mutually independent, then the joint CDF satisfies

$$F_{X_1, \dots, X_N}(x_1, \dots, x_n) = F_{X_1}(x_1) \cdots F_{X_N}(x_N),$$

and the probability mass function satisfies

$$P\{X_1 \leq x_1, \dots, X_N \leq x_N\} = P\{X_1 \leq x_1\} \cdots P\{X_N \leq x_N\}.$$

3.5.1 Numerical Example

Chapter 3 Problem 10

Suppose we know how to generate a random variable from any of the distributions $F_1, \dots, F_n$. Describe how one can generate from the distribution

$$F(x) = \prod_{i=1}^n F_i(x).$$

Each distribution function is defined as $F_i(x) = P\{X_i \leq x\}$. Observe the joint distribution of independent random variables $X_1, \dots, X_n$ is given by

$$F(x) = \prod_{i=1}^n F_i(x) = \prod_{i=1}^n P\{X_i \leq x\} = P\{X_1, \dots, X_n \leq x\} = P\{\max_{1 \leq i \leq n} X_i \leq x\}.$$

Then, to generate a random sample X from $F(x)$, we simply do the following:

1. Generate $X_1, \dots, X_n$ from $F_1, \dots, F_n$.
2. Set $X = \max_{1 \leq i \leq n} X_i$.

4 Methods for Simulating Stochastic Processes

In this section, we consider the simulation of random variables indexed by time (i.e. continuous time stochastic processes). We examine the stochastic process $W(t)$ known as **Brownian motion**. Specifically, a Brownian motion with drift μ and volatility σ is characterized by the following properties:

1. $W(0) = 0$
2. $W(t)$ has independent increments
3. $W(t) - W(s) \sim N(\mu(t-s), \sigma^2(t-s))$ for $0 \leq s < t \leq T$.

If $\mu = 0$ and $\sigma^2 = 1$, then the process is known as a standard Brownian motion.

4.1 Brownian Bridge Construction

Suppose we want to generate values $W(t_1), \dots, W(t_n)$ of a Brownian Motion for a fixed time mesh $0 = t_0 < t_1 < \dots < t_n = T$. We can simulate the Brownian Motion using the **Brownian Bridge Construction**. To construct any value $W(s)$ given $W(t_{n-1}) = x_{n-1}$ and $W(t_n) = x_n$ for $s \in (t_{n-1}, t_n)$, Lemma 3.5 [1] states that the conditional distribution of $W(s)$ is a standard normal distribution with

$$\mu = \mathbb{E}[W(s) | W(t_{n-1}) = x_{n-1}, W(t_n) = x_n] = \frac{x_{n-1}(t_n - s) + x_n(s - t_{n-1})}{t_n - t_{n-1}},$$

$$\sigma = \text{Var}(W(s) | W(t_{n-1}) = x_{n-1}, W(t_n) = x_n) = \frac{(t_n - s)(s - t_{n-1})}{t_n - t_{n-1}}.$$

We use the above values to recursively construct a Brownian path.

4.1.1 Numerical Example

Chapter 3 Problem 15

Let $0 = t_0 < t_1 < \dots < t_n = T$. Discuss how you would simulate $W(t)$ backwards in time, i.e., given $W(0) = 0$, simulate the Brownian motion in this order:

$$W(t_n), W(t_{n-1}), \dots, W(t_1).$$

You need to provide equations similar to (3.10).

We first generate $Z_1 \sim N(0, 1)$. Then $W(t_n) = \sqrt{t_n}Z_1$ and Lemma 3.5 [1] gives us the mean and variance for generating $W(t_{n-1})$ given $W(t_n)$ and $W(0) = 0$, where

$$\mu = \frac{0(t_n - t_{n-1}) + W(t_n)(t_{n-1} - 0)}{t_n - 0} = \frac{W(t_n) \cdot t_{n-1}}{t_n},$$

$$\sigma = \frac{(t_n - t_{n-1})(t_{n-1} - 0)}{t_n - 0} = \frac{(t_n - t_{n-1}) \cdot t_{n-1}}{t_n}.$$

Hence $W(t_{n-1}) \sim N(\frac{W(t_n) \cdot t_{n-1}}{t_n}, \frac{(t_n - t_{n-1}) \cdot t_{n-1}}{t_n})$. Generate $Z_2 \sim N(0, 1)$ and set

$$W(t_{n-1}) = \frac{t_{n-1}}{t_n} W(t_n) + \sqrt{\frac{(t_n - t_{n-1}) \cdot t_{n-1}}{t_n}} Z_2.$$

Continuing this process backward in time to t_1 , we obtain

$$W(t_1) = \frac{t_1}{t_2} W(t_2) + \sqrt{\frac{(t_2 - t_1) \cdot t_1}{t_2}} Z_n.$$

We implement this backwards recursion:

```

1  function stochastic_process(T, n)
2      t = range(0, stop=T, length=n+1)
3      W = zeros(n+1)
4      W[n+1] = sqrt(T) * randn()
5      for i in n:-1:2
6          W[i] = (t[i]/t[i+1]) * W[i+1] +
              ↪ sqrt((t[i+1]-t[i])*t[i]/t[i+1]) * randn()
7      end
8      return W
9  end

```

Simulating 100 Brownian paths over time horizon $[0, 1]$ for $n = 10$ time steps, we obtain following:

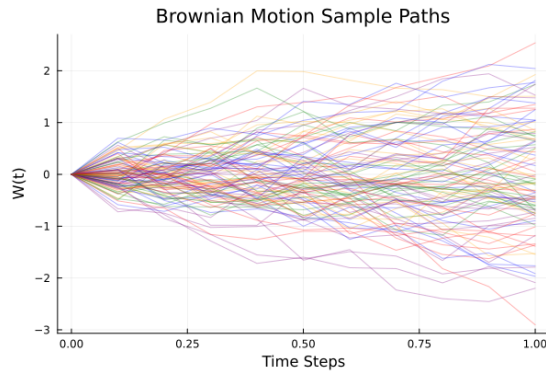


Figure 2: Brownian Motion over $[0, 1]$ with $n = 10$ time steps

Each path above corresponds to a realization of a stochastic process constructed via the Brownian bridge method. All paths originate at $W(0) = 0$ and "fan out" over time, which is consistent with the variance property $\text{Var}(W(t)) = t$. This confirms that our recursive methodology captures the necessary characteristics of Brownian motion.

5 Conclusion

In this report, we explored several fundamental Monte Carlo methods for simulating non-uniform random variables and stochastic processes. The inverse transformation, acceptance-rejection, Box-Muller, and Beasley-Springer-Moro methods were implemented and analyzed both theoretically and numerically.

The numerical results demonstrate each method successfully generates samples consistent with the target distributions. The acceptance-rejection method reproduces discrete distributions, while the Box-Muller and Beasley-Springer-Moro algorithms produce samples consistent with the normal distribution, as shown by the Anderson-Darling test. Furthermore, the simulation of Brownian motion using a recursive "bridge" construction showed the key properties of continuous-time stochastic processes, including independent increments and time-dependent variance.

Overall, this study highlights the effectiveness of using Monte Carlo methods as efficient and practical tools for simulating complex probabilistic systems.

6 Complete Code

```
1  using Random
2  Random.seed!(1234)
3  using Statistics
4  using Plots
5  using LaTeXStrings
6  using Distributions
7
8  # Box-Muller method for generating standard normal random variables
9  function boxmuller()
10     u1 = rand()
11     u2 = rand()
12     r_squared = -2 * log(u1)
13     theta = 2 * pi * u2
14     z1 = sqrt(r_squared) * cos(theta)
15     z2 = sqrt(r_squared) * sin(theta)
16     return z1, z2
17 end
18
19 # Beasley-Springer-Moro method for generating standard normal random
20 ↪ variables
21 function beasley_springer_moro()
22     a = [2.50662823884, -18.61500062529, 41.39119773534,
23         ↪ -25.44106049637]
24     b = [-8.47351093090, 23.08336743743, -21.06224101826,
25         ↪ 3.13082909833]
26     c = [0.3374754822726147, 0.9761690190917186, 0.1607979714918209,
27         ↪ 0.0276438810333863, 0.0038405729373609, 0.0003951896511919,
28         ↪ 0.0000321767881768, 0.0000002888167364, 0.0000003960315187]
29     u = rand()
30     y = u - 0.5
31     if abs(y) < 0.42
32         r = y^2
33         x = y * (((a[4] * r + a[3]) * r + a[2]) * r + a[1]) /
34             ↪ (((b[4] * r + b[3]) * r + b[2]) * r + b[1]) * r + 1)
35     else
36         r = u
37         if y > 0
38             r = 1 - u
39             r = log(-log(r))
40             x = c[1] + r*(c[2] + r*(c[3] + r*(c[4] + r*(c[5] +
41                 ↪ r*(c[6] + r*(c[7] + r*(c[8] + r*c[9]))))))))
42         else
43             r = log(-log(r))
44             x = -(c[1] + r*(c[2] + r*(c[3] + r*(c[4] + r*(c[5] +
45                 ↪ r*(c[6] + r*(c[7] + r*(c[8] + r*c[9]))))))))
46         end
47     end
48 end
```

```

41     return x
42 end
43
44 # Anderson-Darling statistic for testing normality of sample data
45 function anderson_darling_statistic(X)
46     X = sort(X)
47     N = length(X)
48     sum = 0.0
49     for i in 1:N
50         Z = cdf(Normal(0, 1), X[i])
51         sum += ((2*i - 1)*log(Z)) + ((2*N + 1 - 2*i)*log(1 - Z))
52     end
53     A_squared = -N - (sum / N)
54     return A_squared
55 end
56
57 # Acceptance-rejection method for sampling from discrete
58 ↪ distribution
59 function acceptance_rejection()
60     f = [0.3, 0.2, 0.35, 0.15]
61     while true
62         u1 = rand()
63         u2 = rand()
64         y = Int(floor(4*u1)) + 1
65         if u2 <= f[y]/0.35
66             return Int(y)
67         end
68     end
69
70 # Inverse-transformation method for sampling from Weibull
71 ↪ distribution
72 function inverse_transform(alpha,beta)
73     u = rand()
74     x = (-log(u) / alpha)^(1/beta)
75     return x
76 end
77
78 # Brownian bridge method for simulating Brownian paths
79 function stochastic_process(T, n)
80     t = range(0, stop=T, length=n+1)
81     W = zeros(n+1)
82     W[n+1] = sqrt(T) * randn()
83     for i in n:-1:2
84         W[i] = (t[i]/t[i+1]) * W[i+1] +
85             ↪ sqrt((t[i+1]-t[i])*t[i]/t[i+1]) * randn()
86     end
87     return W
88 end

```

```

87
88 function main()
89     X1 = zeros(2000)
90     for i in 1:1000
91         z1, z2 = boxmuller()
92         X1[2*i - 1] = z1
93         X1[2*i] = z2
94     end
95     A1 = anderson_darling_statistic(X1)
96
97     X2 = zeros(2000)
98     for i in 1:2000
99         X2[i] = beasley_springer_moro()
100    end
101    A2 = anderson_darling_statistic(X2)
102
103    println("AD statistic for Box-Muller: ", A1)
104    println("AD statistic for Beasley-Springer-Moro: ", A2)
105
106    x = zeros(1000)
107    for i in 1:1000
108        x[i] = acceptance_rejection()
109    end
110    display(histogram(x, bins=4, normalize=true,
111        ↪ title="Acceptance-Rejection Samples", xlabel="Value",
112        ↪ ylabel="Density", label="Simulated Samples"))
113
114    alpha = 1
115    beta = 1
116    x_inv = zeros(10)
117    for i in 1:10
118        x_inv[i] = inverse_transform(alpha, beta)
119    end
120    println("Generated values from inverse transform: ", x_inv)
121
122    W = zeros(100, 11)
123    t = range(0, stop=1, length=11)
124    for i in 1:100
125        W[i, :] = stochastic_process(1, 10)
126    end
127    display(plot(t, W', title="Brownian Motion Sample Paths",
128        ↪ xlabel="Time Steps", ylabel="W(t)", palette=:rainbow,
129        ↪ alpha=0.3, label=false))
130
131 end
132 main()

```

References

- [1] G. Okten, *Monte Carlo and quasi-Monte Carlo Methods in Computational Finance*, Draft, 2026.
- [2] P. Glasserman, *Monte Carlo Methods in Financial Engineering*, Springer, 2003.
- [3] “Box-Muller transform,” *Wikipedia, The Free Encyclopedia*. Available at: https://en.wikipedia.org/wiki/BoxMuller_transform (Accessed: March 28, 2026).
- [4] “Inverse transform sampling,” *Wikipedia, The Free Encyclopedia*. Available at: https://en.wikipedia.org/wiki/Inverse_transform_sampling (Accessed: March 28, 2026).
- [5] “Rejection sampling,” *Wikipedia, The Free Encyclopedia*. Available at: https://en.wikipedia.org/wiki/Rejection_sampling (Accessed: March 28, 2026).
- [6] “Weibull distribution,” *Wikipedia, The Free Encyclopedia*. Available at: https://en.wikipedia.org/wiki/Weibull_distribution (Accessed: March 28, 2026).